INSTITUTIONAL-GRADE TECHNICAL SPECIFICATION

VeriMind Protocol

AI Attribution & Programmable Royalty Infrastructure

Deployed today on an existing EVM-compatible blockchain. Future direction: verifiable zero-knowledge AI inference, decentralized (DePIN) compute orchestration, and the VeriMind Network / Appchain.

DOCUMENT VERSION:	2.1.0-TECH-SPEC (repository-aligned revision)	**SPECIFICATION TYPE:**	Core Protocol Architecture
DATE:	September 2026	**CHAIN (CURRENT MVP):**	Existing EVM-compatible chain
CHAIN (FUTURE TARGET):	Cosmos SDK + EVM execution (VeriMind Network / Appchain)	**PROVING ENGINE (FUTURE TARGET):**	Halo2 / Plonky3 (SNARK)
NATIVE TOKEN:	$VMIND — 1,000,000,000 fixed supply at deployment	**REPOSITORY:**	github.com/Abanoub5/verimind-protocol

DOCUMENT STATUS & IMPLEMENTATION DISCLOSURE

VeriMind is an early-stage protocol. Its **current MVP is an attribution and programmable royalty infrastructure layer deployed on top of an existing EVM-compatible blockchain** — it is not a Layer-1 blockchain, not a Cosmos SDK application-chain, and does not run its own consensus network. A separate, longer-term direction — a modular Layer-1 combining verifiable zero-knowledge AI inference, a decentralized (DePIN) compute mesh, and an optional dedicated VeriMind Network/Appchain — is specified in this document as **future architecture** and is clearly separated from what exists today. Figures, diagrams, and workflow descriptions in the future-architecture sections reflect design targets and estimates, not measured or achieved results, unless explicitly stated otherwise.

IMPLEMENTED / PROTOTYPED (current MVP)	SPECIFIED (documented target design)	PLANNED / NOT YET BUILT (future architecture)
• 6 Solidity protocol contracts (Section 6) • On-chain royalty settlement logic (RoyaltyManager.sol) • Off-chain vector attribution reference implementation (attribution-engine/attribution.py) • End-to-end prototype/demo (demo/run-demo.js) using a test-only mock verifier • Solidity test suites in test/ (local compile/test run pending in-repo verification)	• Full protocol architecture (this document) • Consensus / validator / slashing model • REST/RPC API specification • ZK circuit design (Halo2/Plonky3)	• Real Halo2/Plonky3 ZK circuits and cryptographic on-chain verification • Cosmos SDK application-chain, CometBFT consensus, live validator set • Networked/decentralized Attribution Node and Compute Node (DePIN) layers • Public testnet or mainnet; live public RPC/API • On-chain token vesting/cliff enforcement • Independent security audit

Current vs. future, in one line — CURRENT: *AI attribution + programmable royalty infrastructure on an existing EVM chain.* ***FUTURE:*** *verifiable AI inference + decentralized compute + the VeriMind Network/Appchain. This distinction is maintained consistently throughout every section below.*

1. Executive Summary & Abstract

VeriMind Protocol addresses three structural gaps in the generative AI economy: the absence of verifiable proof of origin for AI-generated content, the absence of automatic attribution or payment to the data creators whose work shapes a model's output, and the concentration of GPU/TPU compute among a small number of hyperscale providers.

What exists today is a working reference implementation of the first two problems, scoped deliberately narrowly: a vector-based attribution engine that scores dataset creators' contribution to a given output via cosine similarity, and a set of six Solidity smart contracts that turn those attribution scores into enforceable, on-chain programmable royalty settlement on an existing EVM-compatible chain. This MVP does not depend on zero-knowledge proofs, a dedicated consensus network, or decentralized compute — those remain future architecture, described in later sections and clearly labeled as such.

The long-term vision is broader: a modular Layer-1 combining zk-SNARK-verified AI inference with a decentralized (DePIN) compute mesh and dynamic vector attribution, enabling cryptographically-verifiable inference with automated real-time micro-royalty routing. The project's roadmap (Section 12) is deliberately sequenced to validate the attribution-and-royalty product on existing infrastructure before investing in that heavier network-level architecture — summarized by the project's guiding principle: *validate the product before building the network.*

2. Current MVP: AI Attribution & Programmable Royalty Infrastructure

The current MVP is a layer of attribution scoring and programmable royalty logic deployed on top of an existing, already-live EVM-compatible blockchain — it introduces no new consensus mechanism and requires no validator set of its own. Concretely, the MVP consists of:

- Vector-based AI attribution: cosine similarity between an output/query embedding and indexed creator dataset embeddings.
- An attribution scoring engine, implemented as a Python reference implementation (`attribution-engine/attribution.py`), that normalizes similarity into per-creator royalty shares.
- Programmable royalty logic and on-chain royalty settlement, enforced by `RoyaltyManager.sol`.
- Smart-contract based enforcement of escrow, staking, request state transitions, and settlement (Section 6).
- An existing EVM-compatible chain as the deployment target — not a purpose-built Layer-1.
- An end-to-end prototype/demo (`demo/run-demo.js`) that exercises the full flow — escrow, inference lifecycle, attribution, mock ZK verification, and royalty settlement — using a test-only mock verifier in place of real cryptographic proof.

A key architectural boundary in the current MVP: **attribution score calculation is an off-chain / reference implementation** (Python), while **royalty rule enforcement and settlement execution are on-chain** (Solidity). The two are bridged in the demo by `demo/attribution_bridge.py`, which feeds reference-implementation scores to the on-chain settlement contracts. No decentralized network of independently operated Attribution Nodes exists yet — that network topology is future architecture (Sections 4–5, 12).

3. Smart Contract Architecture

The six contracts below are implemented in Solidity and correspond directly to `contracts/` in the repository. They represent the current on-chain reference implementation and are distinct from the live, networked consensus and DePIN layers described in Sections 4 and 12, which remain future architecture.

Contract	Path	Primary Function & Current Status

VMINDToken.sol	contracts/VMINDToken.sol	ERC-20 token, burnable (`ERC20Burnable`). **Does not implement a minting function or a MINTER_ROLE** — the token is not mintable post-deployment. Total/initial supply of 1,000,000,000 VMIND is issued and the initial allocation distributed at deployment time; supply can only move in the downward direction, via burning.
StakingManager.sol	contracts/StakingManager.sol	Manages node deposits/collateral, cooldown logic, and slashing execution at the smart-contract level for the current MVP's actors.
EscrowVault.sol	contracts/EscrowVault.sol	Holds client gas/fee funds during the request lifecycle prior to settlement.
InferenceManager.sol	contracts/InferenceManager.sol	Handles request submission, task state, and proof receipt — currently checked against the test-only `MockZKVerifier`, not a real proof system.
RoyaltyManager.sol	contracts/RoyaltyManager.sol	Processes submitted attribution scores and routes on-chain micro-royalty payments to creators (see Section 7).
Governance.sol	contracts/Governance.sol	Minimal proof-of-concept protocol voting/parameter/treasury logic — not a production DAO governance system (Section 8).
IZKVerifier.sol	contracts/interfaces/IZKVerifier.sol	Verifier interface consumed by InferenceManager — defines the integration boundary for a future real verifier.
MockZKVerifier.sol	contracts/mocks/MockZKVerifier.sol	Test-only implementation of IZKVerifier that returns valid whenever `proof.length > 0`. Performs zero cryptographic verification and must never be treated as a production security mechanism.

Solidity test suites for these contracts exist under `test/`. As of this revision, local `npx hardhat compile` / `npx hardhat test` execution is documented in the repository but has not been independently re-verified in this document; see the Quickstart instructions in `README.md`.

4. Zero-Knowledge Proof of Inference (zk-IE) — Future Architecture

Status: planned, not implemented in the current MVP. *Real zero-knowledge verification of AI inference is a core part of VeriMind's long-term vision and is not removed from this document — but it must not be read as part of the current attribution-and-royalty MVP.*

The current repository's only ZK-related components are: `IZKVerifier.sol` (the verifier interface), `MockZKVerifier.sol` (a test-only mock that performs no cryptographic verification and simply returns `true` when `proof.length > 0`), and `InferenceManager.sol` as the integration point that can call a verifier. The presence of these components does not mean production ZK verification exists today. No real Halo2/Plonky3 verifier, no production ZK circuits, and no cryptographic proof guarantees are implemented as of this revision.

4.1 Target Circuit Design (Future Research)

The target zk-Inference Engine (zk-IE) is designed to translate neural-network forward passes into Plonky3 / Halo2 polynomial constraint systems over a Goldilocks field, with matrix multiplications and non-linear activations (ReLU, GELU) decomposed into PLOOKUP-style lookup arguments to reduce prover overhead. This circuit design has not been built.

Parameter	Target / Estimate	Status
Proving system	Halo2 / Plonky3 (recursive SNARKs)	Planned
Proof size	~1.5–12 KB (compressed, estimated)	Not measured — design target only
On-chain verification latency	<10 ms (estimated, EVM precompile)	Not measured — design target only
Public-input binding (prompt hash, model ID, output commitment)	Interface accepts publicInputs	Not implemented — mock ignores them

No proof size, verification latency, or throughput figure anywhere in this document has been benchmarked, measured, or achieved against a working Halo2/Plonky3 implementation. Before any production deployment of ZK-based inference, the following work is required: (1) define and implement a concrete circuit for at least one reference model architecture; (2) fix the exact public-input/output-commitment format; (3) implement the corresponding production verifier; (4) validate proof generation/verification against independent test vectors; (5) an appropriate security review of circuit and verifier; (6) integration with InferenceManager; (7) removal of any unsecured single-EOA control over verifier configuration in production; and (8) strict confinement of MockZKVerifier to test/development environments only.

Deep activation-layer micro-royalties (attributing royalties to internal model activations rather than only top-level output embeddings) remain an open research direction and are not part of the current attribution reference implementation (Section 5).

5. Consensus, Validators & DePIN Compute — Future Architecture

Status: planned, not operational. *None of the following exist as running systems today: decentralized compute nodes operating as a network, a networked/decentralized Attribution Node layer, a validator set, CometBFT, a Cosmos SDK application-chain, a VeriMind appchain, native protocol-level consensus, or network-level slashing. The technical designs below describe the target architecture and are retained for completeness and grant/investor due diligence, but should not be read as operational infrastructure.*

5.1 Target Consensus Design

The target design proposes a modified CometBFT Proof-of-Stake mechanism with a target 1-second block time and target instant finality, safe under the standard BFT assumption that fewer than 1/3 of total voting weight is malicious. This Cosmos SDK application-chain and CometBFT validator network is a planned component; no mainnet, testnet, or validator set is operating today.

5.2 Target Slashing Parameters

- Double signing (safety violation): target 5% stake slashed, 30-day validator jail — applies once a live validator network exists.
- Liveness fault (downtime): target 0.01% slash for missing 500 of 10,000 consecutive block signatures — applies once a live validator network exists.
- Invalid proof submission (compute slashing): Compute Nodes are designed to forfeit collateral staked in InferenceManager for invalid proof submission; the staking/collateral primitive is prototyped today in StakingManager.sol, but the networked Compute Node role it would apply to is not yet built.

5.3 DePIN Compute Mesh

Target design: independent GPU/TPU operators stake collateral via StakingManager.sol to join as Compute Nodes and execute AI model forward passes plus proof generation. Today, only the staking/collateral contract logic is prototyped; the networked Compute Node software, work-unit distribution, and node discovery layer are planned and not implemented.

6. Attribution Engine & Micro-Royalty Pipeline

6.1 Mathematical Model

Given an input query vector x ∈ R^d and indexed dataset vector representations D = {d_1, d_2, …, d_N}, the engine calculates top-K cosine similarity:

```
Sim(x, d_i) = (x · d_i) / (||x|| · ||d_i||)
```

Attribution scores are then normalized via a softmax with temperature τ:

```
s_i = exp( Sim(x, d_i) / τ ) / Σ_j exp( Sim(x, d_j) / τ )
```

Normalized scores are subsequently expressed as basis points (summing to 10,000) for on-chain settlement (Section 7).

6.2 Implementation Status

A reference implementation of this cosine-similarity attribution and royalty-scoring algorithm exists today at `attribution-engine/attribution.py` and runs against creator-indexed embeddings, feeding the on-chain royalty settlement logic in Section 7. This is a **correctness / reference implementation**, not a production-scale, decentralized attribution network — Attribution Nodes are not currently decentralized, and no independently-operated node network exists (see Section 5).

Dynamic vector attribution and deep activation-layer micro-royalties remain an active research direction; the current implementation operates on top-level embedding spaces only.

7. RoyaltyManager: On-Chain Settlement Logic

Attribution scores produced by the reference engine (Section 6) are submitted to the on-chain settlement layer implemented in `RoyaltyManager.sol`, which enforces the following on submission:

- Submitted attribution scores must be normalized; the sum of basis points across all creators in a request must equal 10,000.
- Creator addresses must be valid, non-zero addresses.
- Duplicate creator entries within the same settlement request are rejected.
- Replay protection is enforced via requestId settlement tracking, preventing the same request from being settled twice.
- Royalty distribution to creators is executed on-chain once these checks pass.

Trust boundary: *RoyaltyManager enforces the arithmetic and structural validity of submitted scores — it does not cryptographically prove that a given attribution score is the correct output of the attribution algorithm for a given inference. Attribution truth/consensus (i.e., that the submitted scores accurately reflect the underlying similarity computation) remains a trust boundary in the current MVP, resting on the authorized off-chain reference implementation rather than on-chain or cryptographic verification.*

8. Security Model, Trust Boundaries & Risk Analysis

VeriMind's current MVP is **not fully trustless**. The following trust boundaries apply to the system as implemented today:

- Attribution calculation is off-chain (the Python reference implementation), not verified on-chain.
- Royalty settlement relies on an authorized settlement role submitting attribution scores to RoyaltyManager.
- MockZKVerifier provides no cryptographic security and is strictly test-only.
- Administrative roles across the contract suite are privileged (e.g., parameter configuration, verifier wiring).

- Governance.sol is a minimal proof-of-concept, not production DAO governance with the safeguards a live protocol would require.

What is enforced on-chain today:

- Escrow authorization and fund custody (EscrowVault.sol)
- Request state transitions (InferenceManager.sol)
- Settlement conditions and royalty arithmetic (RoyaltyManager.sol)
- Duplicate-creator and replay protection on settlement requests
- Staking, cooldown, and slashing logic where implemented (StakingManager.sol)

Sybil resistance, fake-proof prevention via SNARK hardness, and multi-node consensus on vector retrieval are target properties of the future networked architecture (Sections 4–5) and do not apply to the current single-operator prototype.

The current MVP has undergone security review and testing of its implemented smart contracts and core protocol logic. The review covered access control, authorization boundaries, escrow handling, royalty distribution, staking and slashing logic, replay protection, and other identified security-sensitive paths.
The current MVP should nevertheless be considered experimental and not production-ready for high-value deployments. Security assumptions and implementation details may evolve as the protocol moves toward production deployment and future ZK, DePIN, and network components.

9. Tokenomics, Emission & Governance Structure

Total supply: **1,000,000,000 $VMIND**, issued at deployment. As stated in Section 3, `VMINDToken.sol` has no minting function or MINTER_ROLE — supply cannot increase post-deployment, and can only decrease via burning (ERC20Burnable).

Allocation	Share	Documented Vesting Terms
Community & DePIN Node Rewards	**40%**	Intended emission over 10 years via logarithmic decay — tied to the future DePIN/network vision, not the current MVP.
Ecosystem & Creator Grants	**20%**	Intended linear vesting over 4 years.
Seed & Strategic Investors	**15%**	Intended 1-year cliff, 24-month linear vesting.
Core Team & Contributors	**15%**	Intended 1-year cliff, 36-month linear vesting.
Protocol Treasury & Liquidity	**10%**	Intended to be unlocked at TGE for market stability.

***Important:** the allocation percentages above are the documented policy, unchanged from the protocol specification — but **on-chain vesting and cliff enforcement is not implemented** in the current MVP. There is no on-chain mechanism today that automatically locks or releases these allocations on the schedules described; they represent a documented allocation policy rather than a currently enforced mechanism. Community & DePIN Node Rewards specifically are tied to the future network/DePIN vision (Sections 5, 12) and are not part of the current attribution-and-royalty MVP.*

10. Developer SDK, REST/RPC & API — Planned API Specification

*The endpoint below documents a **planned API specification** for inference submission, matching the request/response contract of the current prototype's InferenceManager reference implementation and MockZKVerifier. **It is not a live, publicly hosted network endpoint** — there is no public API server operating today.*

```
POST /v1/inference/submit Content-Type: application/json { "model_id": "verimind-llm-7b-v1",
"prompt_hash": "0x8f3a...", "max_fee": "5000000", "signature": "0x1b2c..." } RESPONSE: {
"request_id": "req_998241", "status": "PROCESSING", "assigned_node": "0x44a1..." }
```

11. Benchmarking Methodology, Limitations & Future Directions

No production benchmarks — ZK proving/verification or otherwise — have been run or published as of this document's date. Any figures elsewhere in this document (proof size, verification latency, block time, throughput) are engineering **targets and estimates only**, not achieved results. Benchmarking will be conducted via automated test harnesses evaluating latency (TTFT), proof-generation throughput, and verification gas cost once a working ZK proving implementation exists. A known limitation for future work is proving overhead for very large (>100B parameter) models, which motivates future research into recursive Nova-based accumulation schemes.

12. Roadmap

The roadmap below matches `docs/ROADMAP.md` in the repository and intentionally prioritizes validating and deploying the attribution-and-royalty MVP on existing EVM infrastructure before introducing zero-knowledge inference, decentralized compute, or a dedicated Layer-1.

Phase 1 — MVP Foundation (Q3–Q4 2026)

Validate the core attribution and royalty model: vector-based attribution reference implementation, attribution scoring engine, programmable royalty settlement logic, attribution-to-Solidity integration, end-to-end prototype demonstration, and contract test suite — largely complete; remaining work includes hardening the MVP contract architecture and preparing deployment to an existing EVM-compatible network.

Phase 2 — Existing-Chain MVP (Q4 2026–Q1 2027)

Deploy MVP contracts to an existing EVM-compatible test network, integrate attribution records with on-chain royalty settlement, enable creator/dataset registration, and establish initial pilot integrations.

Phase 3 — Production Attribution Infrastructure (2027)

Production-grade, scalable attribution service; application and model-provider integrations; royalty accounting/reporting; economic model validation; security hardening and external review; ecosystem partnerships.

Phase 4 — Verifiable AI Inference (Future)

ZK proof-of-inference research, Halo2/Plonky3 circuit prototype, proof generation/verification benchmarks, on-chain verification architecture, and integration with attribution records. Intentionally deferred until the attribution and royalty infrastructure is validated.

Phase 5 — Decentralized Compute (Future)

Compute node specification and prototype, node registration/staking model, workload orchestration, node incentives, and a decentralized compute testnet.

Phase 6 — VeriMind Network / Appchain (Long-Term)

Evaluate whether a dedicated execution environment is justified; Cosmos SDK architecture prototype, validator/consensus design, network RPC interfaces, native network economics, a dedicated testnet, independent security audits, and mainnet readiness assessment. A dedicated VeriMind network is a long-term architectural direction, not a prerequisite for the current MVP.

*Guiding principle: **validate the product before building the network.** VeriMind will first prove that attribution and programmable royalty settlement provide meaningful value to AI applications, creators, and data contributors. Only after that foundation is validated will the project expand into verifiable inference, decentralized compute, and eventually a dedicated VeriMind network.*

13. Repository Consistency

This document is aligned with, and should be read alongside, the following paths in `github.com/Abanoub5/verimind-protocol` (main branch):

- README.md — project overview, pillar-by-pillar status table, Quickstart
- docs/ROADMAP.md — the six-phase roadmap reproduced in Section 12
- docs/security/zk-security.md — the ZK trust-boundary documentation underlying Section 4
- contracts/ — VMINDToken.sol, StakingManager.sol, EscrowVault.sol, InferenceManager.sol, RoyaltyManager.sol, Governance.sol, interfaces/IZKVerifier.sol, mocks/MockZKVerifier.sol
- attribution-engine/attribution.py — the reference implementation underlying Section 6
- demo/run-demo.js and demo/attribution_bridge.py — the end-to-end prototype referenced in Section 2
- test/ — Solidity contract test suites referenced in Section 3

No file paths beyond those documented in the repository are referenced in this whitepaper.

14. Summary of Changes From v2.0

This revision replaces every claim that implied a live, decentralized, or cryptographically verified system with accurate current-MVP language, while preserving the long-term architecture as clearly labeled future work. In particular, this revision no longer describes, as current state: a live Modular Layer-1; a live Cosmos SDK or CometBFT chain; a live validator set; a live DePIN network; production ZK verification; a mintable VMIND token; a live public RPC/API; trustless or decentralized attribution; or an audit status inconsistent with the repository. Where any of these described a genuine future-architecture element, the underlying idea was kept and explicitly re-labeled as planned/target architecture rather than removed.

15. Academic & Protocol References

1. Ben-Sasson, E., et al. (2018). Scalable, transparent, and succinct computational zero-knowledge proofs (STARKs). IACR Cryptol. ePrint Arch.

2. Kwon, J., & Buchman, E. (2016). Cosmos: A Network of Distributed Blockchains. Whitepaper.

3. Bünz, B., et al. (2020). Recursive Proof Composition without a Trusted Setup. IACR Cryptol. ePrint Arch.